M4010104 / M4020104: PROGRAMMING IN PYTHON · DIGITAL UNIVERSITY KERALA · CAPSTONE PROJECT SPECIFICATION

Project 18: Multi-Decade Indian State Crop Yield & Fertilizer/Pesticide Efficiency Auditor

Effort Budget: 10–12 Hours | Core Stack: Pure Python, NumPy, Pandas, Matplotlib, Seaborn, Plotly | Max Marks: 25 | Instructor: Dr. Sunil TT

1. Executive Overview & Domain Context

Covering over two decades of agricultural production (1997–2020) across Indian states and cropping seasons (Kharif, Rabi, Summer, Whole Year), this macro-agricultural dataset tracks cultivated area, total production, annual rainfall, fertilizer consumption, and pesticide application. Auditing agricultural productivity per unit input is central to assessing food security and sustainable farming.

Your team will build a multi-decade agro-economic modeling suite in pure Python that cleans crop production records, guards against zero-area division errors, computes fertilizer efficiency ratios (kg yield per kg fertilizer) in NumPy, fits log-linear Cobb-Douglas production functions, and benchmarks state-level sustainable farming answering the 12 core questions below.

Primary Public Dataset & Telemetry/Data Schema

The project utilizes the verified public Agricultural Crop Yield in Indian States (1997–2020) on Kaggle (**`https://www.kaggle.com/datasets/akshatgupta7/crop-yield-in-indian-states-dataset`**). The dataset structure captures:

- **Raw Feature Attributes:** **`Crop`**, **`Crop_Year`**, **`Season`**, **`State`**, **`Area`** (ha), **`Production`** (tonnes), **`Annual_Rainfall`** (mm), **`Fertilizer`** (kg), **`Pesticide`** (kg), **`Yield`**.
- **Engineered / Target Variables:** **`fertilizer_intensity`** (kg/ha), **`pesticide_intensity`** (kg/ha), **`yield_per_fertilizer`**, **`log_yield`**, **`efficiency_tier`**.

2. Complete Project Directory & File Architecture

Each team must organize their repository into a standardized, production-grade layout. All application source code must reside inside a dedicated package (**`src/`**), with separate directories for raw/curated data, interactive development notebooks, visual figures, and formal documentation:

```
indian-crop-yield-efficiency-auditor/
├── README.md                          # Repository documentation & GitHub Pages source
├── requirements.txt                   # Dependency specification (numpy, pandas, matplotlib, seaborn, p
├── run_pipeline.py                    # Standalone production CLI application entry point
├── data/
│   ├── raw/
│   │   └── crop_yield_indian_states_raw.csv           # Raw uncleaned input dataset (including simulat
│   └── processed/
│       ├── clean_crop_yield_states.csv          # Curated, validated clean CSV dataset
│       └── crop_yield_audit.log           # Timestamped audit report detailing filtered/rejected record
├── src/
│   ├── __init__.py                    # Python package initialization
│   ├── ingestion.py                   # Defensive line-by-line file reader, range auditor & CSV exporte
│   ├── analyzer.py                    # AgriEfficiencyAuditor OOP class, NumPy vectorization & Pandas a
│   └── visualizer.py                  # Matplotlib envelope plots, Seaborn heatmaps & Plotly HTML expor
├── notebooks/
│   └── development_cycle.ipynb        # Development cycle notebook: EDA, trials, tests & unit assertion
└── reports/
    ├── figures/
    │   ├── primary_envelope.png       # Matplotlib confidence envelope & regression overlay
    │   ├── correlation_heatmap.png    # Seaborn annotated feature correlation heatmap
    │   └── distribution_regimes.png   # Seaborn distribution across categorical/domain tiers
```

```
├── interactive_dashboard.html     # Standalone interactive Plotly HTML dashboard with controls
└── Project_Report.pdf             # 3-4 page final technical report
```

3. Dual Development Deliverables: Notebook vs. Standalone Program

To mirror industry software engineering standards, each student group must deliver two distinct, complementary implementations of their solution:

Deliverable A: The Development Cycle Notebook (`notebooks/development_cycle.ipynb`)

The development notebook chronicles your team's engineering journey from raw data exploration to validated model prototypes. It is not merely a final script pasted into cells, but an authentic record of the development cycle:

- **Phase 1: Exploratory Data Ingestion & Diagnostics:** Loading raw data, inspecting missing values, testing boundary rules, and logging rejected records.
- **Phase 2: Mathematical Formulation & Feature Verification:** Developing and validating NumPy array calculations against target benchmarks using `np.allclose()`.
- **Phase 3: OOP Class Architecture & Unit Tests:** Instantiating the `AgriEfficiencyAuditor` class, executing automated unit assertions (e.g., verifying that passing invalid parameters raises an explicit `ValueError`), and testing edge cases.
- **Phase 4: Statistical Modeling & Regimen Exploration:** Implementing ordinary least-squares regression, rolling-window calculations, and multi-metric Pandas grouping.
- **Phase 5: Visual Prototype Development:** Iteratively designing and refining Matplotlib and Seaborn figures, followed by configuring the interactive Plotly HTML dashboard.

Deliverable B: The Final Standalone Python Program (`run_pipeline.py` & `src/`)

While the notebook documents the exploratory development cycle, your repository must also provide a production-ready, fully executable Python application. The program must run end-to-end directly from the terminal without requiring Jupyter:

- **Command-Line Interface (CLI):** Executed via `python run_pipeline.py --input data/raw/crop_yield_indian_states_raw.csv --output-dir data/processed --report-dir reports`.
- **Modular Package Import:** The driver script imports cleanly separated domain logic from `src.ingestion`, `src.analyzer`, and `src.visualizer`.
- **Terminal Feedback & Exit Codes:** The program must print structured progress indicators, log clean/faulty record counts to the console, and return an exit code of `0` upon successful pipeline completion.
- **Automated Artifact Generation:** In a single execution pass, the script reads raw inputs, writes `data/processed/clean_crop_yield_states.csv`, saves `data/processed/crop_yield_audit.log`, exports static figures to `reports/figures/`, and generates the standalone `reports/interactive_dashboard.html`.

4. Technical Investigation Requirements & Research Questions (Q1–Q12)

Your team's implementation across the development notebook and standalone program must rigorously answer the following 12 course-aligned engineering questions:

Course Topic & Script	Q#	Concrete Engineering & Analytical Task	Python Libraries & Methods
File I/O & Exceptions (`src/ingestion.py`)	**Q1**	**Agricultural Range & Division-by-Zero Audit:** Parse multi-decade crop yield CSV rows using try-except. Guard against zero or negative cultivation Area, filter invalid Crop_Year bounds [1997, 2020], and log corrupt records.	open(), try-except (ValueError, ZeroDivisionError), range checks

Course Topic & Script	Q#	Concrete Engineering & Analytical Task	Python Libraries & Methods
Data Curation & Export (`src/ingestion.py`)	**Q2**	**Curated State Crop Yield Export & Audit Log:** Write validated records to clean_crop_yield_states.csv while logging crops with zero production or missing fertilizer data in crop_yield_audit.log.	Defensive file writing, dictionary parsing, formatted log reporting
OOP Design (`src/ingestion.py`)	**Q3**	**Agri Efficiency Auditor Class:** Design an encapsulated class AgriEfficiencyAuditor with methods load_from_csv(), compute_yield_elasticity(crop='Rice'), and summary_report(). Ensure invalid crop raises ValueError.	OOP class, __init__, encapsulation, raise ValueError
NumPy Arrays (`src/analyzer.py`)	**Q4**	**Vectorized Production Function & Verification:** Using only raw NumPy arrays, compute log-transformed variables (log(Area), log(Fertilizer), log(Rainfall)) and verify calculated Yield against CSV Yield via np.allclose().	np.log(), np.divide(), np.allclose()
NumPy Masking (`src/analyzer.py`)	**Q5**	**High Fertilizer Intensity vs. Low Yield Masking:** Flag state-crop records exhibiting high fertilizer intensity (> 200 kg/ha) but below-average yield in NumPy. Identify whether diminishing returns are evident across rice-growing states.	np.mean(), np.std(), boolean masks (&, |), Python set
NumPy Regression (`src/analyzer.py`)	**Q6**	**Cobb-Douglas Crop Production OLS Regression:** Fit a log-linear Cobb-Douglas model (log(Yield) = b0 + b1*log(Fertilizer/ha) + b2*log(Rainfall)) via np.linalg.lstsq. Quantify marginal fertilizer and rainfall elasticity.	np.column_stack(), np.linalg.lstsq(), matrix product (@), R2
Pandas Series (`src/analyzer.py`)	**Q7**	**State-Wise Temporal Completeness & Crop Audit:** Quantify annual crop records for Kerala and South Indian states in Pandas. Audit the frequency of major crops (Rice, Coconut, Tapioca, Sugarcane) over the 24-year window.	.groupby(['State', 'Crop']).count(), .isna().sum()
Pandas Groupby (`src/analyzer.py`)	**Q8**	**Season-by-State Discretization & Pivot Table:** Discretize records into Kharif, Rabi, and Whole Year. Build a State-by-Season pivot_table(margins=True) displaying mean yield per hectare for Rice and Coconut.	pd.pivot_table(margins=True), .groupby().agg()
Pandas Reshaping (`src/analyzer.py`)	**Q9**	**Wide-to-Long Agricultural Input Ranking via Melt:** Reshape input metrics (Fertilizer_kg, Pesticide_kg, Area_ha) into tidy long format via pd.melt() and extract top 5 states with the highest input usage per crop.	pd.melt(), .groupby(['Crop', 'Input_Type']), .nlargest(5)
Matplotlib (`src/visualizer.py`)	**Q10**	**Matplotlib Fertilizer vs. Yield Diminishing Returns:** Plot Fertilizer Intensity (kg/ha) vs. Crop Yield (tonnes/ha) with Matplotlib. Overlay the OLS log-linear regression curve showing diminishing marginal returns with custom legends.	plt.subplots(), ax.scatter(), ax.plot(), custom legends
Seaborn (`src/visualizer.py`)	**Q11**	**Seaborn Multi-Decade State Heatmap & Boxplots:** Render an annotated year-by-state median yield heatmap with sns.heatmap(). Create a side-by-side boxplot comparing fertilizer efficiency across South Indian states.	sns.heatmap(), sns.boxplot(), sns.despine()

Course Topic & Script	Q#	Concrete Engineering & Analytical Task	Python Libraries & Methods
Plotly HTML (`src/visualizer.py`)	**Q12**	**Interactive State Agricultural Yield Dashboard:** Build an interactive HTML dashboard (interactive_dashboard.html) displaying crop yield trends with state dropdowns, crop selectors, and input intensity sliders.	plotly.express.line() / scatter(), hover tooltips, write_html()

5. Collaborative Git Workflow, GitHub Pages & README.md Specification

Best Practices: Collaborative Git Workflow & GitHub Pages Deployment from Day 1

From the very first day, the student team must adopt professional version control and documentation practices. On Day 1, one designated team member initializes the central public GitHub repository, establishes a clear branch protection rule preventing direct pushes to `main`, adds all teammates as registered **Contributors** (with write access), and grants read access to the Course Instructor (`Dr. Sunil TT`) and teaching assistants. Immediately thereafter, the team must activate **GitHub Pages** (via *Settings → Pages → Build and deployment: Deploy from a branch: main /docs or root*) so that project documentation, interactive architecture diagrams, and the live Plotly dashboard (`reports/interactive_dashboard.html`) are publicly accessible and continuously deployed throughout the development lifecycle. All feature development must follow a strict branching model (e.g., `feature/ingestion`, `feature/analysis`, `feature/visualization`): individual team members must write small, atomic, well-commented commits from their own personal GitHub accounts and merge them into `main` exclusively through formal **Pull Requests (PRs)** accompanied by peer review. Dumping entire codebases in a single, monolithic end-of-semester commit is strictly prohibited and will result in zero marks for individual Git contribution; maintaining an unbroken, transparent Git commit and PR audit trail is essential for passing the code ownership defense during the live viva-voce examination.

Repository README.md Specification Template

The root `README.md` serves as the primary technical landing page and must strictly conform to the following structural format:

````
# Project 18: Multi-Decade Indian State Crop Yield & Fertilizer/Pesticide Efficiency Auditor
[![Course](https://img.shields.io/badge/Course-Programming%20in%20Python-blue.svg)](https://duk.ac.in)
[![Python](https://img.shields.io/badge/Python-3.11%2B-brightgreen.svg)](https://python.org)
[![License](https://img.shields.io/badge/License-MIT-yellow.svg)](LICENSE)

## 1. Project Overview & Domain Context
- **Institution:** Digital University Kerala (MTech AI / MSc AI)
- **Course:** M4010104 / M4020104 – Programming in Python | **Instructor:** Dr. Sunil TT
- **Dataset:** Agricultural Crop Yield in Indian States (1997–2020) on Kaggle (https://www.kaggle.com/da

## 2. System Architecture & Pipeline Design
[Provide an ASCII or Mermaid architecture flow diagram showing raw ingestion -> validation -> analyzer -

## 3. Installation & Environment Setup
```bash
git clone https://github.com/<your-team>/indian-crop-yield-efficiency-auditor.git
cd indian-crop-yield-efficiency-auditor
python -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate
pip install -r requirements.txt
```

## 4. Execution Instructions
### A. Running the Standalone Application
```bash
python run_pipeline.py --input data/raw/crop_yield_indian_states_raw.csv --output-dir data/processed --r
```
### B. Running the Development Notebook
```bash
````
````

```
jupyter notebook notebooks/development_cycle.ipynb
```

5. Key Analytical & Statistical Findings (Q1-Q12 Summary)
[Provide a concise markdown summary table detailing raw vs. clean counts, regression coefficients, anoma

6. Interactive Telemetry Dashboard
Explore the live interactive Plotly visualization hosted on GitHub Pages:
🔗 [Live Project Dashboard](https://<your-team>.github.io/indian-crop-yield-efficiency-auditor/interacti

7. Contributor Git Commit & Pull Request Log Table
Contributor Name	GitHub Handle	Assigned Module / Tasks	Verified Commit Hash Range	Merged Pull
[Student Full Name]	`@github_handle`	Modular Architecture (`src/ingestion.py`, `src/analyzer.py`,

6. Evaluation Rubric & Mandatory Academic Integrity Governance

Projects are evaluated on a 25-mark scale based on code quality, mathematical correctness, visualization depth, git collaboration discipline, and live technical defense:

Assessment Criterion	Basis of Evaluation & Deliverable Verification	Max Marks
1. Data Curation & File I/O Pipeline (Q1–Q2)	Robust line-by-line file reading with defensive **`try-except`** handling, physical/domain bounds validation, clean CSV output, and structured **`crop_yield_audit.log`** reporting.	**5 Marks**
2. OOP, NumPy & Pandas Analytics (Q3–Q9)	Modular **`AgriEfficiencyAuditor`** class architecture, vectorized NumPy feature re-derivations (verified via **`np.allclose`**), regression modeling, and Pandas rolling/groupby/melt operations.	**6 Marks**
3. Multi-Library Visualizations (Q10–Q12)	Publication-quality Matplotlib confidence envelope/trend plot, Seaborn correlation heatmap and distribution violin plot, and standalone interactive Plotly HTML dashboard.	**5 Marks**
4. Software Packaging, GitHub & Documentation	Clean directory layout, comprehensive **`README.md`**, working GitHub Pages site, **`development_cycle.ipynb`** notebook, standalone **`run_pipeline.py`**, and balanced individual Git commits/PRs.	**4 Marks**
5. Live Presentation & Code Viva-Voce Defense	Technical walkthrough and individual live viva defense where each student must explain or modify lines of code in real time without external assistance.	**5 Marks**

> **STRICT ACADEMIC INTEGRITY & ZERO-TOLERANCE AI POLICY (0 MARKS PENALTY):** All submitted Python source code (**`.py`**, **`.ipynb`**) and written documentation will be screened through an automated **AI-Code & Plagiarism Detection Suite**. If AI-generated code or synthetic documentation is detected at any checkpoint or final submission, **0 Marks will be awarded immediately** for the entire project. Each student must prove authentic individual code ownership during the live viva-voce examination — *Course Instructor: Dr. Sunil TT*.

7. Milestone Schedule, Submission Checklist & Viva Defense Prompts

To ensure systematic progress across the semester, teams must adhere to three project checkpoints and prepare for on-the-spot live code modification challenges during the viva-voce examination:

Milestone Stage	Scheduled Window	Required Deliverables & Verification Milestones
Checkpoint 1 (Data Ingestion)	Week 11 Lab Session	Repository initialized with contributors and GitHub Pages active. **`src/ingestion.py`** operational with line-by-line filtering, **`clean_crop_yield_states.csv`** exported, and **`crop_yield_audit.log`** generated.

Milestone Stage	Scheduled Window	Required Deliverables & Verification Milestones
Checkpoint 2 (Analytics & OOP)	Week 13 Lab Session	**`src/analyzer.py`** complete with **`AgriEfficiencyAuditor`** class, NumPy feature checks (Q4), outlier masks (Q5), regression modeling (Q6), and exploratory notebook sections.
Final Submission & Viva	Week 15 Project Review	Full package operational: standalone **`run_pipeline.py`**, polished **`development_cycle.ipynb`**, interactive **`interactive_dashboard.html`** on GitHub Pages, 3–4 page PDF report, and live code defense.

Representative Live Viva-Voce Code Modification Challenge

During the individual code viva-voce, the instructor will ask students to execute an immediate 2-minute code modification on their working repository to demonstrate genuine authorship:

- **Challenge Prompt:** *"In src/analyzer.py, filter for Kerala state and compute the yield-per-kg-fertilizer trend for Rice vs. Coconut over the decade 2010-2020. Re-run your module and display the results."*